CYBERSECURITY PORTFOLIO PROJECT

PhishGuard v2

An Educational Email Phishing Awareness Tool

TYPE
Web Application

LANGUAGE
Python 3.10+

FRAMEWORK
Streamlit ≥ 1.32

DOMAIN
Cybersecurity / Education

EXTERNAL APIS
None

ML / AI
None (rule-based)

PhishGuard v2 is a beginner-friendly web application that helps users learn to recognise phishing emails through transparent, explainable pattern-based analysis. It accepts real `.eml` file uploads or pasted email text, runs 10 clue checks, and displays structured findings — what was detected, why it matters, and a safe action to take — along with a risk score and a downloadable Markdown report. **No email content is ever stored, logged, or transmitted.**

Table of Contents

1. Problem Statement
2. Project Objectives
3. Target Users
4. System Architecture & How It Works
5. Features
6. Analysis Engine — Deep Dive
7. Technologies Used
8. Privacy & Safety Design
9. Testing Performed
10. Limitations
11. Future Improvements
12. Conclusion

---

1. Problem Statement

Phishing remains one of the most effective and widespread social-engineering attacks in cybersecurity today. Attackers craft deceptive emails that trick recipients into revealing passwords, financial details, or personal information — or into clicking malicious links and downloading malware.

Many people — especially those new to cybersecurity — do not know what specific clues to look for in a suspicious email. Professional email security tools (spam filters, secure email gateways) do exist, but they:

- Operate **invisibly** — decisions are made with no explanation.
- Give **no educational feedback** — users learn nothing.
- Require **server infrastructure** the average learner does not have.

> **The gap PhishGuard fills:** There is a difference between reading a tip sheet about phishing and actually *practising* recognition skills. PhishGuard bridges that gap with an interactive, explainable tool that shows *which* clues triggered an alert and *why* each one is suspicious — making learning active rather than passive.

2. Project Objectives

1. Let users provide an email by uploading a `.eml` file or pasting text directly.
2. Parse `.eml` files using Python's standard-library `email` package and extract useful headers (From, Reply-To, Subject) and the body.
3. Run transparent keyword-and-pattern checks and present findings with plain-English explanations (what was detected, why it matters, a safe action).
4. Detect and report a From / Reply-To domain mismatch when both headers are present.
5. List all URLs found in the email as plain text only — never fetch, visit, or render them.
6. Report attachment filenames found in `.eml` files and advise the user accordingly.
7. Show a risk estimate (Low / Medium / High) with a clear colour-coded visual treatment.
8. Provide a downloadable plain-text / Markdown report of the findings.
9. Show one randomly chosen phishing-awareness tip per session, kept stable throughout.
10. Organise the experience into focused tabs so the interface is not a single long form.
11. Ensure no email content is saved, logged, or transmitted beyond the current session.

3. Target Users

🎓 Cybersecurity Students & Beginners

Students in bootcamps or self-study programs who want hands-on practice identifying phishing emails before entering the workforce.

🏢 Security Awareness Trainees

Employees completing basic security awareness training who benefit from an interactive tool that explains phishing clues in plain English.

🌐 General Internet Users

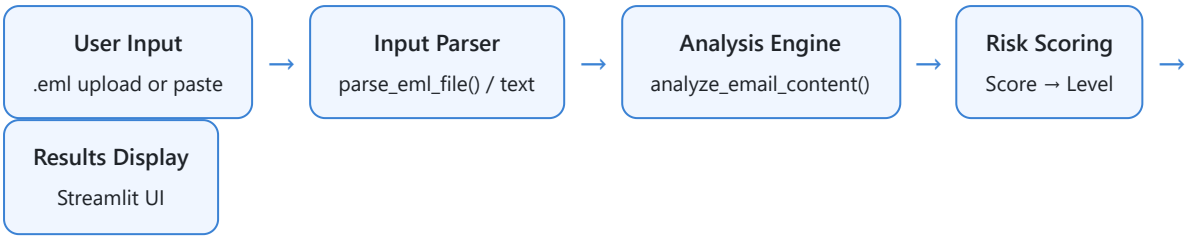
Anyone who wants to strengthen their personal phishing recognition skills without needing any technical background.

📚 Educators & Instructors

Teachers and trainers looking for a live classroom demonstration tool that visualises how phishing emails are analysed.

4. System Architecture & How It Works

4.1 High-Level Data Flow



All processing happens locally in the Streamlit server process. No data leaves the machine.

4.2 Three-Tab User Interface

Tab	Purpose	Key Components
📩 **Provide Email**	Input stage — upload a .eml file or paste email text with optional headers	File uploader, text area, From / Reply-To / Subject fields, demo sample selector, conflict-resolution radio
🔍 **Review Findings**	Analysis results — risk card, structured findings, URL list, download button	Colour-coded risk card, score progress bar, findings accordion, attachment list, URL code blocks
📖 **Learn More**	Education — explains phishing, scoring, clue categories, limitations, next steps	Expandable sections covering all clue definitions, awareness tips, recommended actions

4.3 Input Handling

.eml file upload: The file is read as raw bytes, validated against a 200 KB size limit, then parsed with Python's `email.message_from_bytes()` using `policy.default` for clean Unicode output. The parser walks all MIME parts, collects the plain-text body (falling back to a stripped HTML body if no plain text is present), records attachment filenames, and extracts From, Reply-To, and Subject headers.

Pasted text: The body text is used directly. Header fields (From, Reply-To, Subject) are optional and entered separately in dedicated input fields.

Conflict resolution: If both a file and pasted text are provided simultaneously, the app presents a radio button asking the user to choose — no silent override occurs.

---

5. Features

Feature	Description
.eml File Upload	Upload a real .eml file (max 200 KB). Headers and body are extracted using Python's standard email library.
Paste Email Text	Type or paste a plain-text email body with optional From, Reply-To, and Subject headers for richer analysis.
10 Pattern-Based Checks	Covers urgent language, password requests, financial requests, personal info requests, URLs, IP-address links, URL shorteners / lookalike domains, generic greetings, attachments, and authority impersonation.
Sender / Reply-To Mismatch	Flags when From and Reply-To addresses belong to different domains — a common attacker trick to intercept replies.
URL Listing (Safe)	All URLs are extracted and displayed as plain code blocks — never clickable hyperlinks.
Attachment Detection	Filenames from .eml attachments are listed; high-risk extensions (.exe, .zip, .docm, .js, .bat, .vbs) score higher.
Risk Card	Colour-coded card (Low / Medium / High) with numeric score and progress bar.
Structured Findings	Each finding shows: what was detected, why it may matter, and a recommended safe action.
Downloadable Report	A Markdown report (.md) containing the risk estimate and findings — email body excluded by default for privacy.
Four Demo Samples	Fictional Low, Medium, and two High-risk sample emails that demonstrate the tool safely without real threats.
Rotating Session Tip	One randomly selected phishing-awareness tip displayed at session start and kept stable throughout.
HTML Email Handling	HTML-only emails are safely stripped to plain text — never rendered as live page content.
Learn More Tab	Expandable educational sections covering phishing basics, scoring, clue categories, limitations, and recommended next steps.
Privacy Notice	Prominent expandable banner at the top advising users not to upload confidential real emails.

6. Analysis Engine — Deep Dive

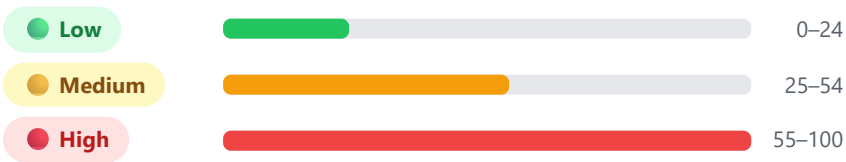
6.1 The 10 Clue Categories

The core function `analyze_email_content()` concatenates the From address, Reply-To address, Subject, and body into a single searchable string, then runs every pattern in `CLUE_DEFINITIONS` as a case-insensitive `re.search()`. Each match adds its defined point value to a running total.

#	Clue Category	Points	What It Detects
1	Urgent / Threatening Language	25	Words like "urgent", "immediately", "suspended", "verify now", "final notice"
2	Password / PIN Request	30	Phrases asking the user to provide, reset, or confirm a password or PIN
3	Financial / Payment Request	30	Requests for credit card numbers, bank details, wire transfers, or payments
4	Sensitive Personal Information	25	Requests for SSN, date of birth, full name, address, or identity documents
5	Links Present	10	Any http:// or https:// URL present in the email
6	IP-Address Link	35	URLs where the host is a raw IPv4 address (e.g. http://192.168.1.1/login)
7	URL Shortener / Lookalike Domain	35	Known shortener services (bit.ly, tinyurl) or visually similar brand domains
8	Generic / Impersonal Greeting	15	"Dear Customer", "Dear User", "Hello Friend" — absence of the recipient's real name
9	Attachment Mentioned or Present	20	Text mentioning attachments, or actual attachments detected in .eml file
10	Authority Impersonation	20	Claims to be IT support, a bank, HMRC, a delivery service, or a tech company

6.2 Risk Score Mapping

The total score is capped at 100 and mapped to a risk level:



6.3 Sender / Reply-To Mismatch

The `check_sender_reply_to_mismatch()` function extracts the domain after `@` in both the From and Reply-To headers. If both are present and the domains differ, a separate warning is displayed above the findings. This is a common attacker technique to ensure replies go to the attacker rather than the spoofed legitimate sender.

6.4 Attachment Scoring

When a `.eml` file is uploaded, actual attachment filenames are extracted from MIME parts. File extensions considered high-risk (`.exe` , `.zip` , `.docm` , `.xlsm` , `.js` , `.bat` , `.vbs`) add 30 points; other extensions add 15 points per attachment.

6.5 URL Extraction & Display

All URLs in the email body are extracted with `re.findall()` and displayed using Streamlit's `st.code()` blocks — never as clickable hyperlinks — so the user can read and evaluate them without accidentally visiting a malicious site. PhishGuard makes **no network requests** of any kind.

```
Score Calculation Summary
──────────────────────────────────────
  Each matched clue  →  +N points
  Actual risky attachment  →  +30 pts each
  Actual other attachment  →  +15 pts each
  Total score capped at    →  100
──────────────────────────────────────
  0–24   →  🟢 Low Risk
  25–54  →  🟡 Medium Risk
  55–100 →  🔴 High Risk
```

---

7. Technologies Used

Tool / Technology	Version / Source	Role in Project
Python	3.10+	Core programming language for all logic
Streamlit	≥ 1.32	Web application framework — UI, tabs, session state, file upload, download button
email + email.policy	Standard library	Parsing .eml files and extracting MIME parts, headers, and body text
re (regex)	Standard library	All pattern-matching for phishing clue detection
html	Standard library	Safely converting HTML entity codes to plain text
random	Standard library	Selecting the rotating phishing-awareness tip at session start
datetime	Standard library	Timestamping the downloaded analysis report
textwrap	Standard library	Text formatting utilities used in report generation

Zero external APIs. All processing happens locally. No network calls are made, no third-party services are contacted, and no user data leaves the running process. **Zero ML / AI dependencies.** All rules are explicit, readable, and easily understandable by a beginner without any background in machine learning.

---

8. Privacy & Safety Design

No Data Storage

Uploaded bytes and pasted text exist only in the Streamlit server process for the current session. All data is discarded when the browser tab is closed.

No HTML Rendering

Uploaded HTML email bodies are never rendered as live page content. The `extract_text_from_html()` function strips all tags and treats the result as plain text only.

No Attachment Execution

Attachments are never opened, executed, or scanned. Only filenames are recorded and reported to the user as plain text.

No Network Requests

URLs found in emails are listed as plain text only. PhishGuard never fetches, visits, or previews any link in the email content.

Privacy Notice

A prominent expandable privacy banner at the top of the app advises users not to upload real confidential emails and to close the tab on shared computers.

Redacted Report

The downloadable report excludes the full email body by default. Users are warned that subject lines or domain names may appear in findings snippets.

---

9. Testing Performed

The following manual test cases were designed and executed to verify the application behaves correctly across all main user journeys.

#	Test Case	Expected Result
1	Empty input — click Analyse with no input provided	Info message prompts user to provide input. No crash.
2	Low-risk fictional sample (newsletter)	🟢 Low Risk, score below 25, only "Links present" clue (or none) triggered.
3	High-risk sample — account suspension scam with pre-filled headers	🔴 High Risk, score ≥ 55. Clues include urgent language, password request, financial request, IP-address link, generic greeting.
4	Medium-risk sample — prize notification with Reply-To mismatch	🟡 Medium Risk. Separate mismatch warning displayed above findings.
5	High-risk sample — fake parcel delivery with bit.ly link	🔴 High Risk. URL shortener, urgent language, payment request, authority impersonation clues triggered.
6	Conflict resolution — file uploaded AND text pasted	Warning appears asking user to choose input source. Radio button shown.
7	Oversized file — .eml file larger than 200 KB	Error message explaining the size limit. No crash.
8	Download report after analysis	phishguard_report.md downloaded with risk level, score, and findings. Email body not included.
9	Learn More tab — expand all sections	All expandable sections open correctly and display content.
10	Tip stability — run analysis and click buttons repeatedly	The session tip does not change during the session.

10. Limitations

> **Important:** PhishGuard is an *educational learning aid*, not a security product. A Low risk score does **not** mean an email is safe.

- **Simple pattern matching is not foolproof.** A sophisticated phishing email that avoids the checked keywords will score Low.
- **No URL fetching or DNS lookup.** PhishGuard cannot visit links or check where they lead; it only inspects the URL text.
- **English only.** All detection patterns are written for English-language emails only.
- **No cryptographic header verification.** The app cannot check SPF, DKIM, or DMARC records, which are required to detect email spoofing reliably.

- **No attachment scanning.** Attached files are never opened, executed, or scanned for malware.
- **False positives are possible.** Legitimate emails (real password resets, bank statements, IT support emails) may trigger clues.
- **No persistence.** All analysis results and session data are lost when the browser tab is closed.
- **.eml files only.** Other formats (.msg, raw MBOX) are not supported.
- **HTML-only emails analysed with reduced accuracy.** The HTML-to-text conversion may strip content that could be relevant to scoring.

---

11. Future Improvements

Improvement	Description
Header Deep-Dive	Parse and display Received headers, X-Originating-IP, and DKIM signature status for technical users.
Lookalike Domain Detection	Use edit-distance heuristics to detect domains similar to a configurable list of trusted brands (not just a hard-coded list).
Multi-Language Support	Extend pattern sets to detect phishing in languages other than English.
Larger Sample Library	Add more fictional examples covering spear-phishing, CEO fraud, and invoice-redirect scams.
Session History	Show a list of analyses run in the current session so users can compare risk scores across emails.
Accessibility Improvements	Better screen-reader labels, keyboard navigation, and high-contrast mode support.
.msg Support	Accept Outlook .msg files in addition to .eml format.
Configurable Scoring	Let users adjust the weight of each clue category to explore how the overall score changes.

---

12. Conclusion

PhishGuard v2 demonstrates how Python's standard library and Streamlit can be combined into a practical, transparent, and genuinely educational security tool. The redesigned three-tab interface — *Provide Email*, *Review Findings*, *Learn More* — keeps the user journey focused and avoids overwhelming beginners with a single long form.

Key improvements over a basic v1 implementation include: proper `.eml` MIME parsing, structured three-part findings (what / why / safe action), sender/Reply-To mismatch detection, safe URL listing as plain text, attachment filename reporting, a downloadable Markdown report, and a rotating session awareness tip.

The project covers core Python concepts — functions, regular expressions, dictionaries, MIME parsing, and Streamlit's session state and component model — while remaining readable enough for a beginner to understand, modify, and extend.

> Most importantly, PhishGuard is **honest about what it can and cannot do**: every finding explains its reasoning, and the application consistently reminds users that its results are educational indicators, not security verdicts. This transparency is itself a core teaching goal — modelling the kind of critical thinking users should apply to real-world email security decisions.

All sample emails included in the application are entirely fictional and were created for educational purposes only. No real phishing emails, credentials, or exploit code are used anywhere in this project.
